

Task 8: Iteration and Loops in Python (for & while)

Track: Data Science | Level 1 - Day 8 | Tools: Python, Jupyter Notebook

Level 1 - Day 8: Iteration and Loops in Python (`for` & `while`)

Track: Data Science

Task: Task 8 - Iteration and Control Flow for Batch Data Processing

Objective: Master `for` loops, `while` loops, control statements (`break`, `continue`, `pass`), iteration helpers (`enumerate()`, `zip()`, `range()`), and nested looping across realistic data processing workflows.

1. Quick Syntax & Loop Mechanisms Overview

Loop / Mechanism	Syntax Pattern	Primary Data Science Role
`for ... in`	`for item in sequence:`	Iterating through dataset records, feature columns, and lists
`while`	`while condition:`	Convergence checks, iterative optimization, simulation epochs
`enumerate()`	`for idx, item in enumerate(seq):`	Tracking indices during batch processing and logging
`zip()`	`for a, b in zip(list1, list2):`	Parallel feature alignment and comparison across arrays
`break` / `continue`	`break` / `continue`	Early stopping, anomaly isolation, skipping corrupt records

2. Core Looping Patterns (8 Practical Scenarios)

In [code]

```
# Example 1: Standard 'for' loop over numerical sequences (Aggregation)
# Calculate cumulative sum and sample mean without built-in sum()
daily_temperatures = [24.5, 26.0, 25.5, 28.2, 27.8, 23.9, 25.1]

total_temp = 0.0
for temp in daily_temperatures:
    total_temp += temp

mean_temp = total_temp / len(daily_temperatures)
print(f"Ex 1 - Mean Temperature: {mean_temp:.2f}°C (Total: {total_temp:.2f}°C)")
```

In [code]

```
# Example 2: Tracking Row Indices using enumerate()
# Locate and flag low-inventory threshold indices
stock_levels = [45, 12, 88, 5, 29, 3]
reorder_threshold = 10

for idx, stock in enumerate(stock_levels, start=1):
    if stock < reorder_threshold:
        print(f"Ex 2 - [Row {idx}] Critical Stock Alert: {stock} units remaining (Action: Reorder)")
```

In [code]

```
# Example 3: Parallel Processing with zip()
# Compute prediction errors between actual and predicted target values
y_true = [100, 150, 200, 250, 300]
y_pred = [95, 155, 190, 260, 295]

print("Ex 3 - Prediction Residuals:")
for actual, pred in zip(y_true, y_pred):
    error = pred - actual
    print(f"  Actual: {actual:<3} | Predicted: {pred:<3} | Residual: {error:>+3}")
```

In [code]

```
# Example 4: 'while' loop with Convergence / Tolerance Criteria
# Simulating iterative gradient descent learning until loss converges
loss = 1.0
tolerance = 0.05
learning_rate = 0.35
epoch = 0

while loss > tolerance:
    epoch += 1
    loss = loss * (1 - learning_rate)
    print(f"  [Epoch {epoch}] Current Loss: {loss:.4f}")

print(f"Ex 4 - Converged after {epoch} epochs (Final Loss: {loss:.4f})")
```

In [code]

```
# Example 5: Skipping Corrupted Records using 'continue'
# Filter and skip null/negative values in sensor telemetry stream
raw_telemetry = [42.1, None, -999.0, 43.5, 41.8, "INVALID", 44.0]
clean_readings = []

for reading in raw_telemetry:
    if reading is None or not isinstance(reading, (int, float)) or reading < 0:
        continue # skip invalid record
    clean_readings.append(reading)

print(f"Ex 5 - Cleaned Readings: {clean_readings}")
```

In [code]

```
# Example 6: Early Stopping using 'break'
# Stop scanning dataset once an anomaly threshold is breached
server_latencies = [45, 52, 48, 61, 250, 49, 53] # in milliseconds
max_allowed_latency = 200

for latency in server_latencies:
    if latency > max_allowed_latency:
        print(f"Ex 6 - Critical Spike Detected: {latency}ms! Halting batch intake.")
        break
    print(f"  Latency normal: {latency}ms")
```

In [code]

```
# Example 7: Nested 'for' loop for Matrix Feature Transformation
# Normalize a 2D numerical matrix (3x3 dataset) by dividing by row maximums
matrix = [
    [10, 20, 30],
    [15, 30, 45],
    [25, 50, 100]
]

normalized_matrix = []
for row in matrix:
    row_max = max(row)
    norm_row = []
    for val in row:
        norm_row.append(round(val / row_max, 2))
    normalized_matrix.append(norm_row)

print(f"Ex 7 - Normalized Matrix: {normalized_matrix}")
```

In [code]

```
# Example 8: Loop with 'else' Clause
# Search for a specific target ID; trigger 'else' only if no break occurred
target_id = 999
enrolled_student_ids = [101, 102, 103, 104, 105]

for s_id in enrolled_student_ids:
    if s_id == target_id:
        print(f"Ex 8 - Student ID {target_id} found in active registry.")
        break
else:
    print(f"Ex 8 - Target ID {target_id} was NOT found in the cohort registry.")
```

3. End-to-End Applied Batch Processing Pipelines

Applied iterative pipelines simulating real batch data engineering and data science aggregation on **Student Academic Performance** and **Employee Compensation** datasets.

In [code]

```
# Pipeline 1: Batch Student Performance Evaluator with Nested Grade Loops
student_cohort = [
    {"id": "ST-01", "name": "Aman", "subject_marks": [88, 92, 79], "attendance": 90},
    {"id": "ST-02", "name": "Pooja", "subject_marks": [55, 62, 58], "attendance": 70},
    {"id": "ST-03", "name": "Rohan", "subject_marks": [42, 38, 45], "attendance": 82},
    {"id": "ST-04", "name": "Sneha", "subject_marks": [94, 96, 91], "attendance": 95},
    {"id": "ST-05", "name": "Vikas", "subject_marks": [72, 68, 75], "attendance": 60}
]

print("=== BATCH STUDENT PERFORMANCE EVALUATION ===")
cohort_total_score = 0.0
total_subjects_evaluated = 0

for s in student_cohort:
    # Calculate student mean via inner loop
    student_sum = 0
    for score in s["subject_marks"]:
        student_sum += score
        cohort_total_score += score
        total_subjects_evaluated += 1

    avg = round(student_sum / len(s["subject_marks"]), 2)
    att = s["attendance"]

    # Evaluate outcome
    if avg >= 85 and att >= 80:
        grade = "Distinction"
    elif avg >= 60 and att >= 65:
        grade = "First Class"
    elif avg >= 50:
        grade = "Pass"
    else:
        grade = "Remedial"

    print(f"[{s['id']}] {s['name']:<6} | Avg Score: {avg:>5.2f}% | Att: {att:>2}% | Result: {grade}")

cohort_mean = cohort_total_score / total_subjects_evaluated
print(f"\nOverall Cohort Average Score: {cohort_mean:.2f}%")
```

In [code]

```
# Pipeline 2: Employee Departmental Spend & Iterative Appraisal Engine
employee_records = [
    {"emp_id": "E101", "name": "Aditi", "dept": "Data Science", "salary": 110000, "rating": 4.8},
    {"emp_id": "E102", "name": "Karan", "dept": "Sales Ops", "salary": 45000, "rating": 3.2},
    {"emp_id": "E103", "name": "Neha", "dept": "Data Science", "salary": 82000, "rating": 4.1},
    {"emp_id": "E104", "name": "Rahul", "dept": "Human Resources", "salary": 50000, "rating": 4.5},
    {"emp_id": "E105", "name": "Simran", "dept": "Sales Ops", "salary": 48000, "rating": 3.9}
]

dept_payroll = {}
dept_counts = {}

print("\n=== EMPLOYEE APPRAISAL DISBURSEMENT REPORT ===")
for emp in employee_records:
    sal = emp["salary"]
    dept = emp["dept"]
    rating = emp["rating"]

    # Calculate iterative hike percentage
    if rating >= 4.5:
        hike_pct = 12.0
    elif rating >= 3.8:
        hike_pct = 8.0
    else:
        hike_pct = 3.0

    revised_sal = sal * (1 + hike_pct / 100)

    # Accumulate department payroll
    dept_payroll[dept] = dept_payroll.get(dept, 0.0) + revised_sal
    dept_counts[dept] = dept_counts.get(dept, 0) + 1

    print(f"[{emp['emp_id']}] {emp['name']:<7} | Dept: {dept:<15} | Old: ${sal:,.2f} -> New: $
    {revised_sal:,.2f} ({hike_pct}%)")

print("\n=== DEPARTMENTAL BUDGET ALLOCATIONS ===")
for dept, budget in dept_payroll.items():
    avg_headcount_cost = budget / dept_counts[dept]
    print(f"Dept: {dept:<16} | Headcount: {dept_counts[dept]} | Total Budget: ${budget:,.2f} | Avg
    Cost/Head: ${avg_headcount_cost:,.2f}")
```